

Automata & ANTLR Cheat Sheet

Comprehensive guide covering formal language theory, automata models, grammar formalisms, and ANTLR usage with rich examples.

1. Fundamentals of Formal Languages

1.1 Basic Definitions

- **Alphabet (Σ):** A finite set of symbols. Example: $\Sigma = \{a, b, 0, 1, +, *\}$.
- **String (word):** A finite sequence of symbols from Σ . Example: `abba`, `1010`, `a+bb*`.
- **Empty string (ϵ / λ):** The unique string of length zero.
- **Kleene closure (Σ^*):** The set of all strings (including ϵ) over Σ .
- **Language:** Any subset of Σ^* . Example: $L = \{w \in \{a,b\}^* : w \text{ contains an even number of } a\text{'s}\}$.

1.2 Operations on Languages

- **Union ($L_1 \cup L_2$):** All strings in L_1 or L_2 .
- **Concatenation ($L_1 \cdot L_2$):** All strings formed by x from L_1 followed by y from L_2 .
- **Kleene star (L^*):** Zero or more concatenations of strings from L .
- **Intersection / Complement:** Defined when Σ is fixed, used for closure properties.

1.3 Example: Even-a Language

- $\Sigma = \{a, b\}$
- $L = \{w \mid w \text{ has an even number of } a\text{'s}\}$
- Regular expression: `(b*ab*a)*b*`
- DFA states encode parity: $Q = \{q_{\text{even}}, q_{\text{odd}}\}$, start = q_{even} , accept = $\{q_{\text{even}}\}$.

2. Regular Languages & Regular Expressions

2.1 Regular Expressions Syntax

Operator	Syntax	Meaning	Example
Concatenation	<code>r₁r₂</code>	Strings matching r_1 followed by r_2	<code>ab</code> matches "ab"
Union	<code>r₁ r₂</code>	Strings matching r_1 or r_2	<code>a b</code> matches "a", "b"
Kleene star	<code>r*</code>	Zero or more repetitions of r	<code>a*</code> matches "", "a", "aa"

Operator	Syntax	Meaning	Example
Plus	<code>r+</code>	One or more repetitions ($r r^*$)	<code>a+</code> matches "a", "aa"
Optional	<code>r?</code>	Zero or one occurrence	<code>a?</code> matches "", "a"
Character class	<code>[abc]</code>	Any one of a, b, or c	<code>[0-9]</code> matches any digit
Negation	<code>[^0-9]</code>	Any symbol not in the class	<code>[^ab]</code> matches any except a or b

2.2 Example Patterns

```
# 1. Binary strings containing "101"
(0|1)*101(0|1)*

# 2. Strings of a's and b's with odd length
((a|b)(a|b))* (a|b)

# 3. Floating-point numbers
[+-]?([0-9]+\.[0-9]*|\.[0-9]+)([eE][+-]?[0-9]+)?

# 4. Identifiers in many languages
[a-zA-Z_] [a-zA-Z_0-9]*
```

2.3 From Regex to NFA/DFA

1. **Thompson's construction** builds an NFA from a regex in linear time relative to regex size.
2. **Subset construction** transforms NFA to equivalent DFA (state explosion possible).
3. **DFA minimization** (Hopcroft's algorithm) reduces to minimal number of states.

Example: Regex `(a|b)*abb` → NFA → DFA → Minimized DFA with 4 states recognizing strings ending in "abb".

3. Finite Automata

3.1 Deterministic Finite Automaton (DFA)

- **Definition:** $(Q, \Sigma, \delta, q_0, F)$. Transition $\delta: Q \times \Sigma \rightarrow Q$.
- **Example:** Language of binary strings with even number of 1's.
- $Q = \{q_0 \text{ (even)}, q_1 \text{ (odd)}\}; \Sigma = \{0, 1\}$
- $\delta(q_0, 1) = q_1; \delta(q_1, 1) = q_0; \delta(*, 0) = \text{same state}; \text{start} = q_0; \text{accept} = \{q_0\}$.

3.2 Nondeterministic Finite Automaton (NFA)

- **Definition:** $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$. Accept if any path leads to accepting state.
- **ϵ -transitions** allow spontaneous moves.
- **Subset construction** yields equivalent DFA.

3.3 Pumping Lemma for Regular Languages

- **Statement:** $\exists p$ such that any $w \in L$ with $|w| \geq p$ can be decomposed $w=xyz$ satisfying:
 - $|xy| \leq p$
 - $|y| \geq 1$
 - $\forall i \geq 0, x y^i z \in L$
 - **Use:** Prove non-regularity. Example: $L=\{a^n b^n \mid n \geq 0\}$ is not regular by contradiction assuming pumping.
-

4. Context-Free Grammars (CFG)

4.1 Definitions

- **Grammar** $G=(V,T,P,S)$ where $P: V \rightarrow (V \cup T)^*$.
- **Derivations:** $S \Rightarrow^* w$. Leftmost/rightmost derivations differ for ambiguous grammars.

4.2 Chomsky Normal Form (CNF)

- Productions: $A \rightarrow BC$ or $A \rightarrow a$. Start may go to ϵ .
- **Conversion:** Eliminate ϵ -productions, unit productions, terminals in long productions, then binarize.

4.3 Examples

```
# Balanced parentheses
S → SS | '(' S ')' | ε

# Palindromes over {a,b}
P → 'a' P 'a' | 'b' P 'b' | ε | 'a' | 'b'
```

4.4 Ambiguity & Disambiguation

- **Ambiguous:** Same string has multiple parse trees.
- **Example:** Arithmetic expression grammar:

```
E → E + E | E * E | ( E ) | id
```

- **Unambiguous** with precedence and associativity:

```

E ::= E '+' T | T
T ::= T '*' F | F
F ::= '(' E ')' | id

```

4.5 Pushdown Automata (PDA)

- **Definition:** $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ with stack alphabet Γ . $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2^{\mathbb{N}}(Q \times \Gamma^*)$.
- **Example:** PDA for $\{a^n b^n\}$: Push 'A' for each 'a', pop for each 'b', accept if stack back to Z_0 .

5. Chomsky Hierarchy & Applications

Type	Machine	Grammar Form	Recognizer Complexity
Type-3	FA (DFA/NFA)	Regular Grammar	$O(n)$
Type-2	PDA	Context-Free Grammar	$O(n^3)$ general, $O(n)$ for LL(k)/LR(k)
Type-1	Linear-bounded TM	Context-Sensitive Grammar	PSPACE
Type-0	TM	Recursively enumerable	Undecidable in general

Applications: Lexical analysis (regex $\rightarrow$ DFA), parsing programming languages (LL/LR), natural language processing.

6. ANTLR Overview & Workflow

6.1 What is ANTLR?

- ANother Tool for Language Recognition.
- Generates: Lexer, Parser, Listener, Visitor.
- Supports actions/semantic predicates, error recovery.

6.2 Typical Workflow

1. **Define grammar** in `.g4` file (parser + lexer rules).
2. **Generate code** with `antlr4` tool.
3. **Parse input** to build parse tree.
4. **Traverse** tree using listener or visitor to build AST or interpret.
5. **Semantic actions:** embed actions or use listener/visitor to enforce semantics.

7. Writing ANTLR Grammars (.g4 files)

7.1 Lexer vs Parser Rules

- **Lexer rules:** Uppercase names, define tokens (e.g. `INT : [0-9]+ ;`).
- **Parser rules:** Lowercase, define language structure (e.g. `expr : expr '+' term ;`).

7.2 Common Patterns

```
// Whitespace & comments
WS : [ \t\r\n]+ -> skip ;
LINE_COMMENT : '//' ~[\r\n]* -> skip ;

// Keywords vs Identifiers
PRINT : 'print' ;
ID : [a-zA-Z_] [a-zA-Z_0-9]* ;

INT : [0-9]+ ;
```

7.3 Expression Grammar with Precedence

```
grammar Expr;
prog : stat+ ;
stat : ID '=' expr ';' | 'print' expr ';' ;

expr : <assoc=right> expr '^' expr # Pow
      | expr '*' expr # Mul
      | expr '+' expr # Add
      | INT # Int
      | ID # Id
      | '(' expr ')' # Parens
      ;
```

- `<assoc=right>` sets right-associativity for power operator.

7.4 Multiple Assignment & Lists

```
stat : vars '=' exprs ';' ;
vars : ID (',' ID)* ;
exprs: expr (',' expr)* ;
```

8. Tree Walking Strategies

8.1 Listeners (Event-Driven)

- Generated `BaseListener` with empty enter/exit methods.
- **Approach:** Override `enterRule` or `exitRule` callbacks.
- **Use case:** Logging, building AST with external data structures.

8.2 Visitors (Return-Driven)

- Generated `BaseVisitor<T>` with `visitX(Context)` for each rule.
- **Approach:** Override `visit` methods, return computed values.
- **Use case:** Expression evaluation, AST transformation with return values.

8.3 Example: Expression Visitor (Java)

```
@Override
public Integer visitAdd(ExprParser.AddContext ctx) {
    int left = visit(ctx.expr(0));
    int right = visit(ctx.expr(1));
    return left + right;
}

@Override
public Integer visitInt(ExprParser.IntContext ctx) {
    return Integer.parseInt(ctx.INT().getText());
}
```

9. Error Handling & Debugging

- **Error listeners:** Implement `ANTLRErrorListener` to catch syntax errors.
- **Diagnostic error listener:** `parser.addErrorListener(new DiagnosticErrorListener());`
- **Bail strategy:** `parser.setErrorHandler(new BailErrorStrategy());` to fail fast.
- **Visualizing parse trees:** Use ANTLR's `TestRig (grun)` or integrated IDE plugins.

10. Testing & Tools

- **Unit tests:** Feed input strings, assert parse-tree shapes or visitor results.
- **Grammar testing:** `TestRig` with `-tree`, `-gui`, `-tokens` options.
- **ANTLRWorks/IntelliJ plugin:** Interactive grammar debugging.

11. Performance & Complexity

- **Lexing:** $O(n)$ with DFA-based lexer.
 - **Parsing:** LL(k)/LR(k) linear time $O(n)$ for well-structured grammars.
 - **General CFG parsing:** $O(n^3)$ via CYK or Earley.
-

12. Common Pitfalls & Tips

1. **Left recursion:** Convert to iterative form or use ANTLR's automatic elimination for direct left recursion.
 2. **Grammar ambiguity:** Refactor with precedence, associativity, or rewrite rules.
 3. **Token conflicts:** Order matters; place keywords before identifiers.
 4. **Hidden tokens:** Use channels to skip whitespace/comments without losing positions.
 5. **Performance:** Avoid excessive backtracking—use `parser.addErrorListener` to detect.
-

Expanded cheat sheet based on course material and mock exam, with deeper explanations and richer examples.